Topic: paging

Virtual Memory and Demand Paging

Virtual memory lets a program use more memory than the machine physically has. The program sees a large logical address space, split into fixed-size pages. Physical memory is split into frames of the same size. A page table maps each page to a frame.

With demand paging, a page is only loaded into memory when the program actually touches it. Pages that are never used are never loaded, so more programs fit in memory at once and the CPU stays busier.

Each page table entry has a valid bit. Valid means the page is in memory and the entry holds its frame number. Invalid means the page is not in memory, and the entry points to where the page is stored on disk (the backing store). At the start, every entry is invalid.

Topic: page-faults

Page Faults and Their Cost

A page fault happens when a program accesses a page whose valid bit is off. Handling it: (1) the hardware traps to the operating system and the process state is saved; (2) the OS finds the page on disk; (3) the OS picks a free frame, replacing a page if none are free; (4) the page is read from disk into that frame; (5) the page table entry is updated and marked valid; (6) the instruction that faulted is restarted.

Effective access time: if p is the chance of a page fault, then EAT = (1 - p) x memory access time + p x page fault service time. A page fault takes milliseconds because of disk I/O, while a memory access takes nanoseconds, so even a tiny p slows the system down a lot.

Topic: page-replacement

Page Replacement Algorithms

When memory is full, the OS must choose a victim page to evict. If the victim was modified (its dirty bit is set), it must be written back to disk first. Clean pages can simply be dropped, which saves one disk write.

FIFO evicts the page that has been in memory the longest. It is simple but can suffer from Belady's anomaly: giving it more frames can actually cause more page faults.

Optimal evicts the page that will not be needed for the longest time in the future. It gives the fewest faults but is impossible in practice, because the OS cannot see the future. It is used as a benchmark.

LRU (least recently used) evicts the page that has gone unused the longest. It looks at the past instead of the future and does not suffer from Belady's anomaly. Exact LRU needs either a timestamp per page or a stack of pages, which is expensive.

The clock (second-chance) algorithm approximates LRU with one reference bit per page. Pages sit in a circle. On a fault, if the page under the clock hand has its bit set, the bit is cleared and the hand moves on; the first page found with the bit cleared is evicted.

Topic: frame-allocation

Allocating Frames to Processes

The OS decides how many frames each process gets. Fixed allocation gives a process a set number of frames for its whole life. Variable allocation lets that number grow or shrink over time.

With local replacement, a process can only evict its own pages. With global replacement, a process may take a frame from any process, so its performance also depends on what other processes are doing. Fixed allocation only works with local replacement.

Topic: thrashing

Thrashing and How to Prevent It

Thrashing is when processes spend almost all their time swapping pages in and out instead of doing useful work. It happens when processes do not have enough frames for the pages they are actively using. CPU usage drops, and if the OS reacts by starting even more processes, things get worse.

Programs show locality: they reuse recently used memory (temporal locality) and memory near what they just used (spatial locality). The working-set model uses this. A process's working set is the set of pages it used in its last N references. If the total working sets of all processes exceed the number of frames, the OS suspends a process to free up memory.

The page-fault frequency approach watches each process's fault rate instead. If the rate is too high, the process gets more frames; if it is very low, a frame is taken away.